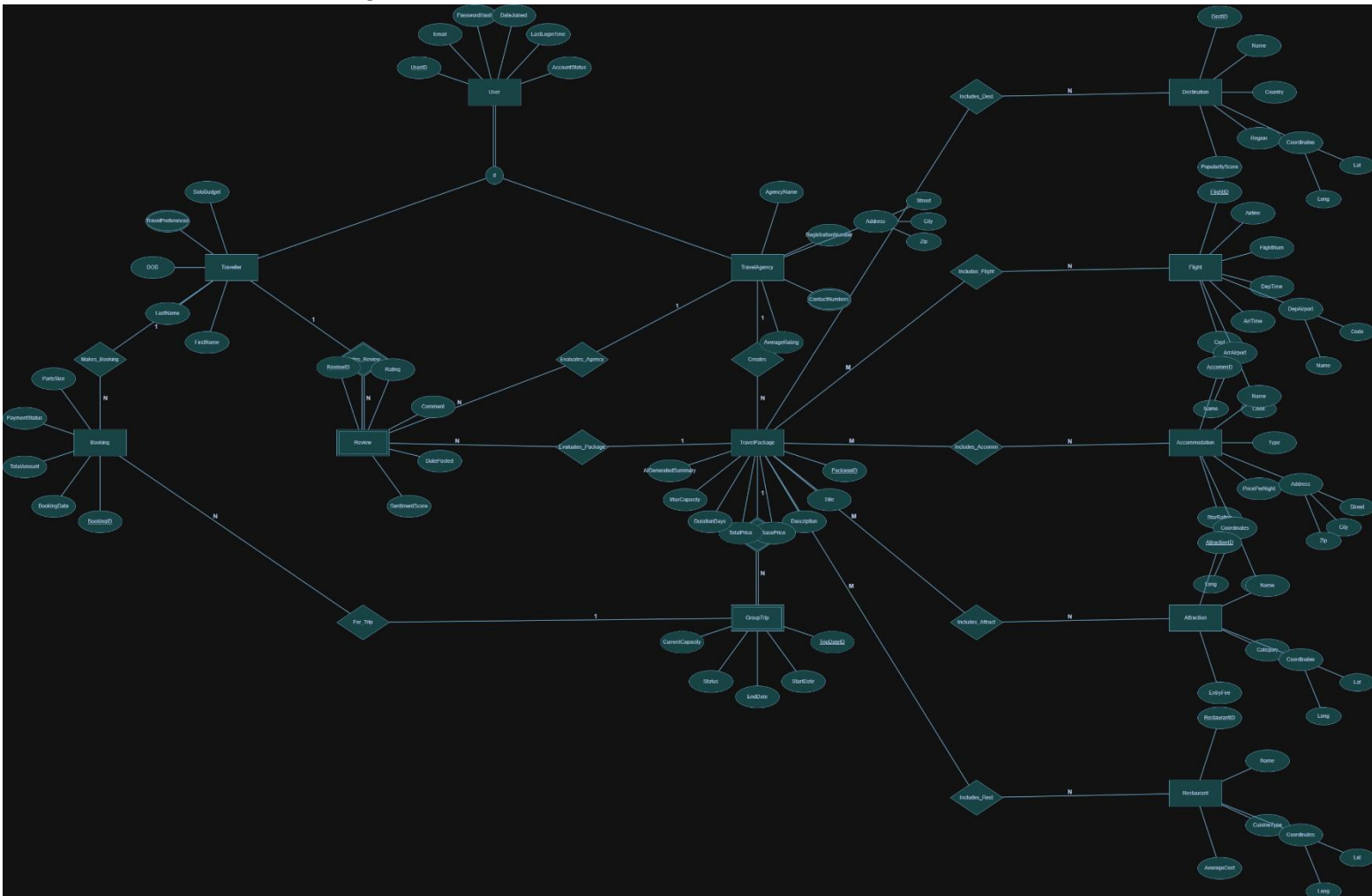


## COS 221 Practical 5 Team 16: The musketeers

Link to repository: <https://github.com/JPmire/COS221-PA5-Tripistry>

### Task 2-EER Diagram:



# Task 3-Relational Mapping:

Step 1: Map Regular Entity Types

|                |                  |                      |                     |                       |                  |                        |                        |                         |                        |              |
|----------------|------------------|----------------------|---------------------|-----------------------|------------------|------------------------|------------------------|-------------------------|------------------------|--------------|
| DESTINATION    | DestID INT       | Name VARCHAR         | Country VARCHAR     | Region VARCHAR        | Lat DECIMAL      | Long DECIMAL           | PopularityScore INT    |                         |                        |              |
| FLIGHT         | FlightID INT     | Airline VARCHAR      | FlightNum VARCHAR   | DepTime DATETIME      | ArrTime DATETIME | DepAirportCode VARCHAR | DepAirportName VARCHAR | ArrAirportCode VARCHAR  | ArrAirportName VARCHAR | Cost DECIMAL |
| ACCOMMODATION  | AccomID INT      | Name VARCHAR         | Type VARCHAR        | PricePerNight DECIMAL | StarRating INT   | Street VARCHAR         | City VARCHAR           | Zip VARCHAR             | Lat DECIMAL            | Long DECIMAL |
| ATTRACTION     | AttractionID INT | Name VARCHAR         | Category VARCHAR    | EntryFee DECIMAL      | Lat DECIMAL      | Long DECIMAL           |                        |                         |                        |              |
| RESTAURANT     | RestaurantID INT | Name VARCHAR         | CuisineType VARCHAR | AverageCost DECIMAL   | Lat DECIMAL      | Long DECIMAL           |                        |                         |                        |              |
| TRAVEL_PACKAGE | PackageID INT    | Title VARCHAR        | Description TEXT    | BasePrice DECIMAL     | DurationDays INT | MaxCapacity INT        | AGeneratedSummary TEXT | TravelAgencyID INT (FK) |                        |              |
| BOOKING        | BookingID INT    | BookingDate DATETIME | TotalAmount DECIMAL | PaymentStatus VARCHAR | PartySize INT    | TravelerID INT (FK)    | PackageID INT (FK)     | TripDateID INT (FK)     |                        |              |

Step 2: Map Weak Entity Types

|            |                     |                  |                |              |                     |                        |                   |                    |  |  |
|------------|---------------------|------------------|----------------|--------------|---------------------|------------------------|-------------------|--------------------|--|--|
| GROUP_TRIP | PackageID INT (FK)  | TripDateID INT   | StartDate DATE | EndDate DATE | Status VARCHAR      |                        |                   |                    |  |  |
| REVIEW     | TravelerID INT (FK) | RestaurantID INT | Rating INT     | Comment TEXT | DatePosted DATETIME | SentimentScore DECIMAL | AgencyID INT (FK) | PackageID INT (FK) |  |  |

### Step 3: Map Binary 1:1 Relationship Types

N/A. No 1:1 relationships are present in this model.

### Step 4: Map Binary 1:N Relationship Types

Foreign keys added to match the 1:N relations natively alongside the entity creation as part of physical DB logic, shown in their respective steps above.

### Step 5: Map Binary M:N Relationship Types

|                       |                           |                              |
|-----------------------|---------------------------|------------------------------|
| PACKAGE_DESTINATION   | <u>PackageID</u> INT (FK) | <u>DestID</u> INT (FK)       |
| PACKAGE_FLIGHT        | <u>PackageID</u> INT (FK) | <u>FlightID</u> INT (FK)     |
| PACKAGE_ACCOMMODATION | <u>PackageID</u> INT (FK) | <u>AccommID</u> INT (FK)     |
| PACKAGE_ATTRACTION    | <u>PackageID</u> INT (FK) | <u>AttractionID</u> INT (FK) |
| PACKAGE_RESTAURANT    | <u>PackageID</u> INT (FK) | <u>RestaurantID</u> INT (FK) |

### Step 6: Map Multivalued Attributes

|                      |                             |                              |
|----------------------|-----------------------------|------------------------------|
| TRAVELLER_PREFERENCE | <u>TravellerID</u> INT (FK) | <u>Preference</u> VARCHAR    |
| AGENCY_CONTACT       | <u>AgencyID</u> INT (FK)    | <u>ContactNumber</u> VARCHAR |

Step 7: Map N-ary Relationship Types

N/A. No N-ary relationships are present in this model.

Step 8: Map Specialization/Generalization

|               |                        |                    |                            |                     |                        |                       |
|---------------|------------------------|--------------------|----------------------------|---------------------|------------------------|-----------------------|
| USER          | <u>UserID</u> INT      | Email VARCHAR      | PasswordHash VARCHAR       | DateJoined DATETIME | LastLoginTime DATETIME | AccountStatus VARCHAR |
| TRAVELLER     | <u>UserID</u> INT (FK) | FirstName VARCHAR  | LastName VARCHAR           | DOB DATE            | SoloBudget DECIMAL     |                       |
| TRAVEL_AGENCY | <u>UserID</u> INT (FK) | AgencyName VARCHAR | RegistrationNumber VARCHAR | Street VARCHAR      | City VARCHAR           | Zip VARCHAR           |

Step 9: Map Union Types

N/A. No Union types are present in this model.

### Task 4-Relational Schema:



## Task 6-Data Sources:

### Data Population Method and Justification

#### 1. Data Sources and Quality

Our group utilized a synthetic, dataset themed around the television shows Ted Lasso, Invincible, and The Boys. While the data is fictional, it mirrors the exact structure, constraints, and data types of real-world travel APIs (such as OpenTripMap or Amadeus) making it effective and sufficient to rigorously test our web application.

- The Destinations and Coordinates use valid latitude and longitude ranges.
- The Packages feature logical durations, varied pricing tiers (budget to luxury), and associated foreign keys that test our complex M:N table joins.
- The thematic nature of the data guarantees distinct, highly recognizable inputs, which significantly improves our ability to debug sorting, filtering, and cross-agency comparisons.

#### 2. Data Entry Method

We populated the database manually using a robust, hand-written SQL script. This method was chosen over automated artificial data generators (like Faker) because our specific database schema contains highly strict constraints and multiple Weak Entities that automated scripts struggle with.

For example:

- Distinct Roles Constraint: We had to manually ensure that Users were exclusively routed into either the TravelAgency or Traveller subclass tables without overlap.
- Weak Entity Sequencing: The Review table has a composite primary key where the ReviewID is a partial key derived from the TravellerID. Manual scripting allowed us to cleanly reset the ReviewID to 1 for each unique traveller, maintaining relational mapping rules.
- Mutually Exclusive Constraints: The script safely navigated the chk\_ReviewTarget constraint, ensuring reviews accurately targeted either an agency or a package, but never both.

By hardcoding this themed SQL script, we ensured our database maintained 100% referential integrity and first-normal-form compliance while perfectly suiting our demo requirements.

## Task 7-Query Optimisation:

### 1. SQL Query for Analysis

We chose to analyze the performance of the core package search query in api.php. This query joins the TravelPackage, TravelAgency, and Package\_Destination tables, filters results dynamically, groups by package ID, and sorts by package base price:

```
EXPLAIN SELECT p.PackageID, p.Title, p.BasePrice, p.DurationDays, p.ImageURL,
a.AgencyName, a.AverageRating FROM TravelPackage p JOIN TravelAgency a ON
p.AgencyID = a.UserID LEFT JOIN Package_Destination pd ON p.PackageID =
pd.PackageID LEFT JOIN Destination d ON pd.DestID = d.DestID GROUP BY
p.PackageID ORDER BY p.BasePrice ASC;
```

### 2. Query Plan Analysis (Before Optimization)

Running this EXPLAIN query in MariaDB reveals a full table scan (type: ALL) on both the TravelPackage (p) and TravelAgency (a) tables. It also shows the use of an in-memory temporary table and filesort (Extra: Using temporary; Using filesort) to group by PackageID and sort by BasePrice. For a small seed dataset (3 rows), this plan is highly efficient. However, as the database grows to thousands of packages, a full table scan and filesort would lead to  $O(N \log N)$  sorting overhead and severe disk-write latency when the sorting size exceeds the temp table memory limit.

### 3. Optimization Strategy

To optimize this performance, we created a single-column index on the sorting attribute BasePrice in the TravelPackage table:

```
CREATE INDEX idx_package_price ON TravelPackage(BasePrice);
```

### 4. Analysis after Optimization and Discussion

After adding the index, we executed the EXPLAIN statement again. For our current seeded database (with only 3 travel packages), the query execution plan remained identical (type: ALL and filesort). This behavior is expected and correct because modern RDBMS optimizers use cost-based models. With only 3 records, reading the data sequentially is faster than traversing the B-Tree index structure and performing random-access read-ahead operations. In a production-sized database, this index allows the optimizer to retrieve records in a pre-sorted order directly from the index leaf nodes, reducing the sorting complexity from  $O(N \log N)$  to  $O(1)$  read operations and completely eliminating the filesort step.

## Task 11-Bonus Marks:

### 1. AI implementation:

We used the Gemini API to perform sentiment analysis on reviews left on travel packages and agencies. We stored the API key in a .env file +, this was so we could access the same API key from different files as well as so that when we were testing we did not have to manually change the API key across different files and functions as the functions code would stay the same and the only change would be made in the .env file. We then included the .env file in the gitignore file so that when we would commit our changes the API key would not be made public. We did this to prevent key leakage and credential scraping.

I used: `__DIR__ . '/.env'` to find the .env file that should be in the same folder as the current file.

First I had to send the prompt by making a JSON payload, then I used client URLs (via PHP's cURL extension) to specify curl options, set `CURLOPT_SSL_VERIFYPEER` to false for local testing environments, and POST the request to the Gemini API endpoint. Once the API returned the response, we parsed the JSON payload to extract the text output containing the sentiment analysis category and score. The score was then stored in the `SentimentScore` field of the `Review` table, ensuring a fully automated, AI-augmented feedback cycle.